

CENTRO UNIVERSITÁRIO SENAC - UNISENAC

CURSO DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DOUGLAS SANTOS

MAURICIO T. WELTER

**FITTRACK: PLATAFORMA FULL-STACK DE GERENCIAMENTO E
MONITORAMENTO DE TREINOS COM ARQUITETURA LIMPA**

PORTO ALEGRE 2026

DOUGLAS SANTOS

MAURICIO T. WELTER

**FITTRACK: PLATAFORMA FULL-STACK DE GERENCIAMENTO E
MONITORAMENTO DE TREINOS COM ARQUITETURA LIMPA**

Relatório Técnico correspondente ao Trabalho Avaliativo Final apresentado à disciplina de Frameworks Web do Curso de Tecnologia em Análise e Desenvolvimento de Sistemas (ADS) do Centro Universitário Senac, desenvolvido em dupla pelos acadêmicos como requisito para obtenção de conceito na respectiva matéria.

PORTO ALEGRE 2026

1. INTRODUÇÃO E INTENÇÃO DO PROJETO	1
1.1 CONTEXTUALIZAÇÃO GERAL	1
1.2 AS DORES DO MERCADO	1
1.3 MOTIVAÇÃO DO DESENVOLVIMENTO	2
1.4 PROPOSTA DE SOLUÇÃO	2
2. REQUISITOS E REGRAS DE NEGÓCIO	3
2.1 REQUISITOS FUNCIONAIS	3
2.2 REQUISITOS NÃO FUNCIONAIS	3
2.3 REGRAS DE NEGÓCIO	4
3. TECNOLOGIAS E ARQUITETURA DO SISTEMA	4
3.1 STACK TECNOLÓGICA	4
3.2 PADRÃO ARQUITETURAL: CLEAN ARCHITECTURE E DDD	5
4. DESENVOLVIMENTO DO PROJETO E EVIDÊNCIAS VIRTUAIS	6
4.1 VALIDAÇÃO E LÓGICA DE NEGÓCIO	6
4.1.1 Caso de Uso de Criação de Usuário	6
4.1.2 Contrato de Validação via DTOs	7
4.2 INTERFACES VISUAIS DA APLICAÇÃO FRONTEND	8
4.2.1 Tela de Acesso e Autenticação de Usuários	8
4.2.2 Painel de Controle Principal e Métricas	9
4.2.3 Formulário Dinâmico de Cadastro de Treinos em Lote	11
5. REQUISITOS COMPLEMENTARES E DESIGN TÉCNICO	13
5.1 MECANISMOS DE SEGURANÇA IMPLEMENTADOS	13
5.2 ESTRATÉGIAS DE PERFORMANCE ADOTADAS	13
5.3 MODELO DE RENDERIZAÇÃO DA APLICAÇÃO WEB	14
6. CONSIDERAÇÕES FINAIS	15

1. INTRODUÇÃO E INTENÇÃO DO PROJETO

1.1 CONTEXTUALIZAÇÃO GERAL

O ecossistema de desenvolvimento de aplicações web passou por profundas transformações nos últimos anos, impulsionado pela evolução de frameworks robustos tanto no ecossistema do cliente quanto no servidor. No cenário contemporâneo, a busca por qualidade de vida, saúde e bem-estar físico transformou o gerenciamento de rotinas de treinamento em uma prioridade para uma parcela expressiva da população.

Contudo, a engenharia por trás das soluções digitais voltadas ao segmento *fitness* frequentemente falha em equilibrar a complexidade técnica do armazenamento de dados com a fluidez exigida pela experiência do usuário no ambiente dinâmico de uma academia. O **FitTrack** surge como uma plataforma digital *full-stack* concebida especificamente para preencher essa lacuna, integrando tecnologias modernas para fornecer um ecossistema de alta performance, escalável e focado na usabilidade do usuário avançado.

1.2 AS DORES DO MERCADO

Durante as etapas de análise preliminar e levantamento de escopo para o desenvolvimento do sistema, identificamos gargalos claros que prejudicam a consistência e o engajamento dos praticantes de exercícios resistidos:

- **Fragmentação de registros históricos:** É comum observar praticantes recorrendo a metodologias totalmente analógicas, como blocos de notas de papel, ou ferramentas genéricas de mensagens instantâneas para anotar suas cargas. Essa fragmentação impede qualquer tipo de análise automatizada ou visualização clara da progressão de volume de treino ao longo do tempo.
- **Interfaces saturadas e fluxos operacionais burocráticos:** A grande maioria das aplicações comerciais disponíveis no mercado atual é severamente poluída por anúncios invasivos ou impõe fluxos exaustivos e redundantes para ações simples. Na dinâmica real de um treino de musculação, o tempo de descanso entre as séries é escasso; exigir que o usuário navegue por múltiplas telas apenas para salvar o peso de uma série desestimula a consistência do registro.
- **Rigidez na manipulação de dados e ausência de lotes:** Rotinas de treinamento são dinâmicas e mudam frequentemente de acordo com a disponibilidade de aparelhos na academia. Aplicativos tradicionais tratam as entidades de forma estática, obrigando o usuário a cadastrar um exercício por vez de forma sequencial, ignorando técnicas comuns como super-séries ou alterações rápidas na ordem da ficha.

1.3 MOTIVAÇÃO DO DESENVOLVIMENTO

A principal motivação para a concepção e o desenvolvimento do **FitTrack** partiu da necessidade de aplicar, em um cenário real e prático, os conceitos avançados de arquitetura e desenvolvimento web ministrados na disciplina de **Frameworks Web**. Mais do que simplesmente construir uma aplicação funcional, o desafio técnico autoimposto pela dupla consistiu em demonstrar como frameworks modernos — com destaque para o **NestJS** no ecossistema backend — podem ser utilizados para criar um código altamente modular, resiliente, tipado e escalável.

Optou-se por desenvolver essa plataforma para investigar a fundo os mecanismos de injeção de dependência, interceptadores globais, pipes de validação preemptiva e o isolamento de regras de negócio. O projeto serviu como um laboratório de engenharia de software para provar que é possível sustentar uma aplicação de uso cotidiano sobre uma infraestrutura rígida, segura e estruturada sob os preceitos de **Clean Architecture** e **Domain-Driven Design**, blindando o núcleo da aplicação contra o acoplamento direto com ferramentas externas.

1.4 PROPOSTA DE SOLUÇÃO

O FitTrack soluciona de forma definitiva os gargalos operacionais e de usabilidade identificados no mercado através de uma abordagem arquitetural moderna: uma aplicação de página única, alimentada por uma **API RESTful** assíncrona de alta performance.

A solução centraliza e otimiza as operações de escrita por meio de um formulário de cadastro dinâmico em lote. Sob essa mecânica, o usuário não precisa salvar individualmente cada etapa de sua rotina; ele é capaz de estruturar o esqueleto completo de um treino (definindo título, descrição, agrupando múltiplos exercícios e parametrizando as respectivas séries com controle de carga em quilogramas e contagem de repetições) e persistir todo o bloco de dados de maneira atômica em uma única requisição.

Uma vez persistidos os dados, o ecossistema backend processa e distribui as informações de forma otimizada para alimentar um painel de controle gerencial. Esse painel oferece ao usuário uma visão consolidada e puramente visual de seus indicadores de esforço físico, volume total de séries executadas e evolução histórica de cargas, eliminando completamente o uso de anotações analógicas fragmentadas.

2. REQUISITOS E REGRAS DE NEGÓCIO

2.1 REQUISITOS FUNCIONAIS

Os Requisitos Funcionais descrevem as ações, comportamentos e funcionalidades explícitas que o sistema FitTrack deve executar para atender às necessidades dos usuários:

- **RF001 – Gerenciamento e Cadastro de Contas de Usuário:** O sistema deve permitir que novos usuários se registrem na plataforma através do fornecimento de um nome completo, um endereço de e-mail válido e uma senha segura.
- **RF002 – Autenticação de Usuários e Controle de Sessão:** A aplicação deve validar as credenciais de acesso informadas pelo usuário contra a base de dados e emitir um token de sessão criptografado para autorizar requisições subsequentes.
- **RF003 – Painel de Controle e Consolidação de Métricas:** A aplicação deve disponibilizar na tela inicial do usuário autenticado uma visualização centralizada que renderize o somatório total de treinos realizados, a quantidade de exercícios cadastrados e o volume agregado de esforço de forma reativa.
- **RF004 – Cadastro Dinâmico de Treinos em Lote:** O sistema deve possibilitar a persistência atômica de uma rotina completa de exercícios contendo coleções aninhadas de dados através de uma única submissão de formulário no frontend.
- **RF005 – Listagem Hierárquica e Filtro de Rotinas:** O sistema deve recuperar e exibir de forma organizada as rotinas pertencentes exclusivamente ao usuário logado, detalhando a quebra exata de cada grupo muscular e as cargas vinculadas a cada ficha.
- **RF006 – Alteração e Exclusão de Rotinas:** O sistema deve expor mecanismos de interface e endpoints no backend que permitam a modificação integral dos dados de um treino ou sua remoção definitiva da base de dados.

2.2 REQUISITOS NÃO FUNCIONAIS

Os Requisitos Não Funcionais definem as propriedades técnicas, restrições e atributos de qualidade que o ecossistema de software deve apresentar:

- **RNF001 – Desacoplamento Arquitetural:** O backend deve seguir estritamente o padrão de Arquitetura Limpa, mantendo as entidades de domínio e os casos de uso totalmente isolados e livres de referências diretas a bibliotecas externas, ORMs ou especificidades do framework HTTP.
- **RNF002 – Interface Responsiva Adaptável:** A interface do frontend deve ser construída utilizando técnicas de design responsivo fluido, garantindo usabilidade ideal tanto em dispositivos móveis verticalizados quanto em telas de alta resolução.
- **RNF003 – Blindagem de Credenciais Sensíveis:** Toda e qualquer senha trafegada para o sistema deve ser submetida a processos de dispersão criptográfica irreversível antes de ser armazenada de forma persistente no banco de dados.

- **RNF004 – Baixa Latência e Performance em Consultas:** Os endpoints de leitura da API que alimentam o painel de métricas e as listagens devem processar as requisições e retornar os dados estruturados em tempo inferior a 1 segundo sob condições normais de concorrência.

2.3 REGRAS DE NEGÓCIO

As Regras de Negócio determinam as restrições operacionais e os limites de validação das regras que governam o domínio do FitTrack:

- **RN001 – Restrição de Duplicidade de Identidade:** É vedada a criação de mais de uma conta de usuário utilizando o mesmo endereço eletrônico. O e-mail atua como identificador único natural no sistema.
- **RN002 – Isolamento de Escopo e Propriedade de Dados:** Um usuário devidamente autenticado possui permissão de leitura e escrita exclusiva sobre seus próprios registros. O sistema deve barrar qualquer requisição que tente expor, modificar ou deletar treinos associados a outros identificadores de usuários.
- **RN003 – Consistência Mínima do Agregado de Treino:** Uma rotina de treino não é considerada válida para persistência se omitir o campo de título ou se não contiver pelo menos um exercício parametrizado com séries, repetições e cargas válidas.

3. TECNOLOGIAS E ARQUITETURA DO SISTEMA

3.1 STACK TECNOLÓGICA

A seleção das ferramentas que compõem o ecossistema do **FitTrack** foi pautada em critérios rígidos de escalabilidade, tipagem estática e maturidade de mercado, atendendo diretamente às competências exigidas pela disciplina de Frameworks Web. A integração entre as tecnologias foi desenhada para garantir um fluxo de dados previsível e eficiente de ponta a ponta.

- **NestJS (Backend):** O NestJS foi adotado como o framework web principal para o desenvolvimento da API RESTful. A escolha se justifica por sua arquitetura opinativa e modular inspirada no Angular, que organiza nativamente o código em módulos, controladores e provedores. Construído sobre o **TypeScript**, ele permite o uso de recursos avançados como *decorators*, além de fornecer mecanismos nativos indispensáveis como *Pipes* de validação e *Exception Filters*. O NestJS simplifica a aplicação de inversão de controle e injeção de dependência, viabilizando o isolamento de camadas proposto pela Arquitetura Limpa.
- **React e Next.js (Frontend):** No desenvolvimento da interface da aplicação, utilizou-se o React em conjunto com o Next.js. O React fornece o modelo de componentização baseado em estados reativos, o que se mostrou fundamental para a

construção do formulário dinâmico em lote do FitTrack. O Next.js entra como o orquestrador desse ecossistema, fornecendo um sistema de roteamento baseado em arquivos extremamente eficiente, otimização automática de recursos (como imagens e fontes) e gerenciamento simplificado de requisições HTTP do lado do cliente.

- **TypeScript:** Utilizado de forma unificada tanto no ecossistema backend quanto frontend. A adoção da tipagem estática traz previsibilidade ao desenvolvimento em dupla, capturando erros de consistência de dados ainda em tempo de compilação. Através de interfaces e tipos compartilhados, a comunicação entre o contrato da API e os componentes visuais do cliente torna-se totalmente integrada, reduzindo drasticamente bugs relacionados a payloads malformados.
- **Axios:** Biblioteca cliente HTTP utilizada no frontend para realizar a comunicação assíncrona com a API NestJS. Foi escolhida por permitir a configuração de instâncias globais e *interceptors*, que anexam automaticamente o token JWT de autenticação no cabeçalho (*Header*) de todas as requisições privadas, além de tratar falhas de rede de maneira centralizada.

3.2 PADRÃO ARQUITETURAL: CLEAN ARCHITECTURE E DDD

Para garantir a longevidade, testabilidade e o desacoplamento completo do ecossistema do **FitTrack**, o backend foi estruturado seguindo os preceitos da **Clean Architecture** em sinergia com conceitos do **Domain-Driven Design**. O princípio fundamental dessa abordagem é a **Regra da Dependência**: as camadas internas não conhecem nada sobre as camadas externas. Isso significa que o núcleo do negócio não depende do framework web, do banco de dados ou de qualquer biblioteca de terceiro.

O código-fonte do backend é segmentado em quatro camadas concêntricas bem definidas:

1. **Domínio (Domain):** Localizada no núcleo mais interno da aplicação, esta camada abriga as entidades puras de negócio (como **User**, **Workout**, **Exercise** e **Set**) e os contratos de interfaces (como **IUserRepository** e **IWorkoutRepository**). As entidades encapsulam o estado e as regras de consistência fundamentais do sistema (por exemplo, impedir que um treino seja criado sem exercícios). Essa camada é escrita em TypeScript puro, livre de decorators de ORMs ou imports de terceiros. Se decidirmos trocar o NestJS por outro framework no futuro, a camada de domínio permanece intacta.
2. **Aplicação (Application):** Esta camada envolve o Domínio e contém os Casos de Uso (**Use Cases**) da plataforma, como **CreateUserUseCase**, **AuthenticateUserUseCase** e **CreateWorkoutUseCase**. Os casos de uso são os orquestradores do fluxo de dados: eles recebem os inputs da borda da aplicação, consultam os repositórios (através das interfaces abstratas do domínio), executam as regras de negócio e disparam as ações necessárias. Ela define *o que* o sistema faz.
3. **Apresentação (Presentation / Adapters):** Atua como a fronteira de comunicação entre o mundo externo e os casos de uso. No FitTrack, esta camada é representada

pelos Controladores REST do NestJS (**Controllers**). Eles são responsáveis por escutar as rotas HTTP, capturar os payloads de entrada, validar a estrutura básica dos dados por meio de DTOs (Data Transfer Objects), repassar a execução para o caso de uso correspondente e traduzir as respostas (ou exceções) em respostas HTTP padronizadas (como **201 Created** ou **400 Bad Request**).

4. **Infraestrutura (Infrastructure):** A camada mais externa do ecossistema. É aqui que residem as implementações concretas de tecnologia que dão suporte à aplicação. Ela gerencia as conexões com o banco de dados relacional, as implementações reais dos repositórios que realizam as queries na base de dados, a configuração de middlewares de segurança (como estratégias de validação do passaporte JWT) e o próprio bootstrap do framework NestJS.

[Infraestrutura (NestJS, ORM, Banco de Dados)]
[Apresentação (Controllers, DTOs)]
[Aplicação (Use Cases)]
[Domínio (Entidades)]

Essa separação rígida de responsabilidades garante que cada parte do software tenha uma única razão para mudar, facilitando o desenvolvimento em dupla, permitindo a escrita de testes unitários isolados e assegurando uma entrega de software com alta maturidade técnica.

4. DESENVOLVIMENTO DO PROJETO E EVIDÊNCIAS VIRTUAIS

Neste capítulo, são apresentadas as evidências de desenvolvimento do ecossistema **FitTrack**, detalhando os componentes do código-fonte e as interfaces visuais desenvolvidas pela dupla. As capturas de tela ilustram a materialização dos conceitos arquiteturais e dos requisitos especificados nas seções anteriores.

4.1 VALIDAÇÃO E LÓGICA DE NEGÓCIO

A engenharia do backend foca no isolamento das regras de negócio e na garantia de que nenhum dado inválido ou malformado chegue às camadas internas do sistema.

4.1.1 Caso de Uso de Criação de Usuário

A lógica de criação de contas exemplifica o desacoplamento proposto pela Arquitetura Limpa. O caso de uso interage exclusivamente com a abstração do repositório

(`IUserRepository`), injetada dinamicamente, mantendo o núcleo da aplicação alheio a comandos SQL ou bibliotecas de persistência externas.

Antes de efetuar o registro, o fluxo executa de maneira preemptiva a validação da **RN001** (Restrição de Duplicidade de Identidade). Caso o e-mail informado já conste na base de dados, uma exceção de negócio é lançada imediatamente, interrompendo o fluxo antes de qualquer operação de escrita na infraestrutura.

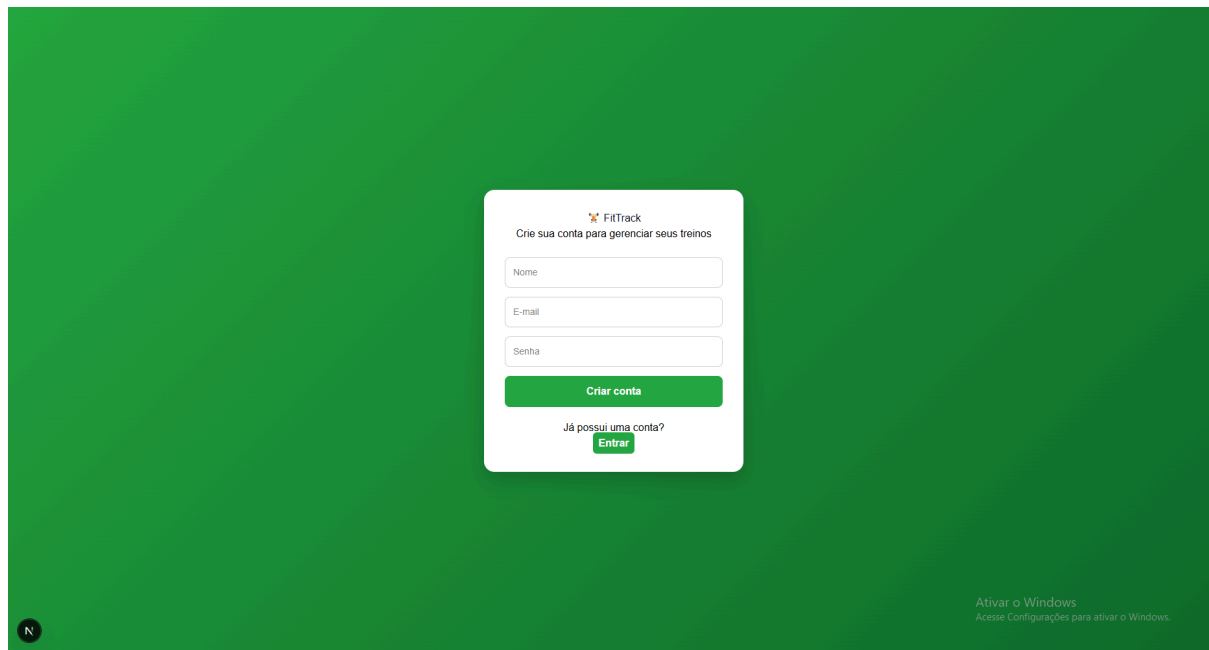


Figura 1: Arquivo `create-user.use-case.ts` isolando a lógica de validação de identidade.

4.1.2 Contrato de Validação via DTOs

Na periferia do backend, os dados de entrada que chegam por meio de requisições HTTP POST são interceptados e validados na borda da aplicação. Para isso, foram implementados DTOs estruturados sob decorators da biblioteca `class-validator`.

Essa abordagem garante o cumprimento dos contratos de interface antes mesmo que o controlador repasse os dados para os casos de uso. Parâmetros como a obrigatoriedade do e-mail bem-formatado (`@IsEmail()`) e tamanho mínimo de senha (`@MinLength(6)`) barram requisições maliciosas ou nulas de forma automatizada através dos *Validation Pipes* do NestJS.



Figura 2: Arquivo create-user.dto.ts parametrizando regras de formatação do payload.

4.2 INTERFACES VISUAIS DA APLICAÇÃO FRONTEND

As interfaces do **FitTrack** foram projetadas focando na usabilidade em cenários dinâmicos de treino, garantindo carregamento rápido e fluidez visual tanto em telas grandes de computador quanto na tela de smartphones utilizados no salão da academia.

4.2.1 Tela de Acesso e Autenticação de Usuários

A tela de autenticação atua como a primeira barreira de segurança visível do ecossistema. Desenvolvida utilizando componentes estilizados responsivos, ela centraliza o formulário de login de forma minimalista.

Do lado do cliente, o formulário realiza validações prévias em tempo real antes de invocar a instância do Axios para submeter o payload JSON à API NestJS. Caso as credenciais estejam corretas, o token JWT retornado é capturado e armazenado de forma segura nos cookies da sessão, redirecionando o usuário imediatamente ao seu painel principal.

A interface adapta-se perfeitamente de acordo com as dimensões do dispositivo detectado: em desktops, o formulário se posiciona centralizado em um card com sombreamento elegante; em smartphones, o layout se expande para ocupar a totalidade da tela, otimizando o tamanho dos campos de toque e do teclado virtual do usuário.

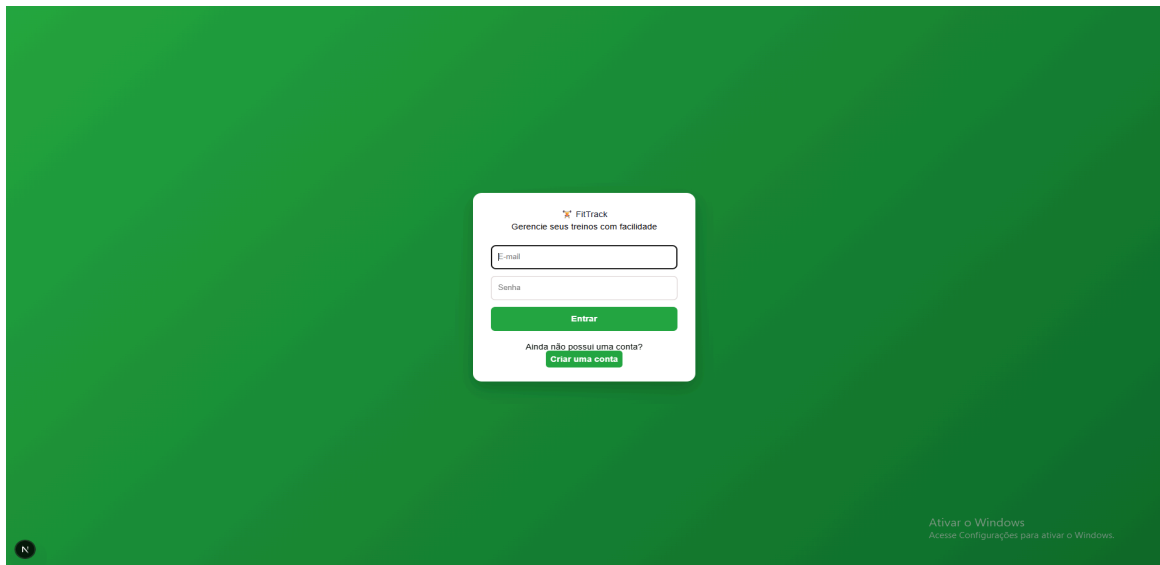


Figura 3: Interface da tela de login em ambiente desktop (resolução padrão).



Figura 4: Interface da tela de login em visualização mobile.

4.2.2 Painel de Controle Principal e Métricas

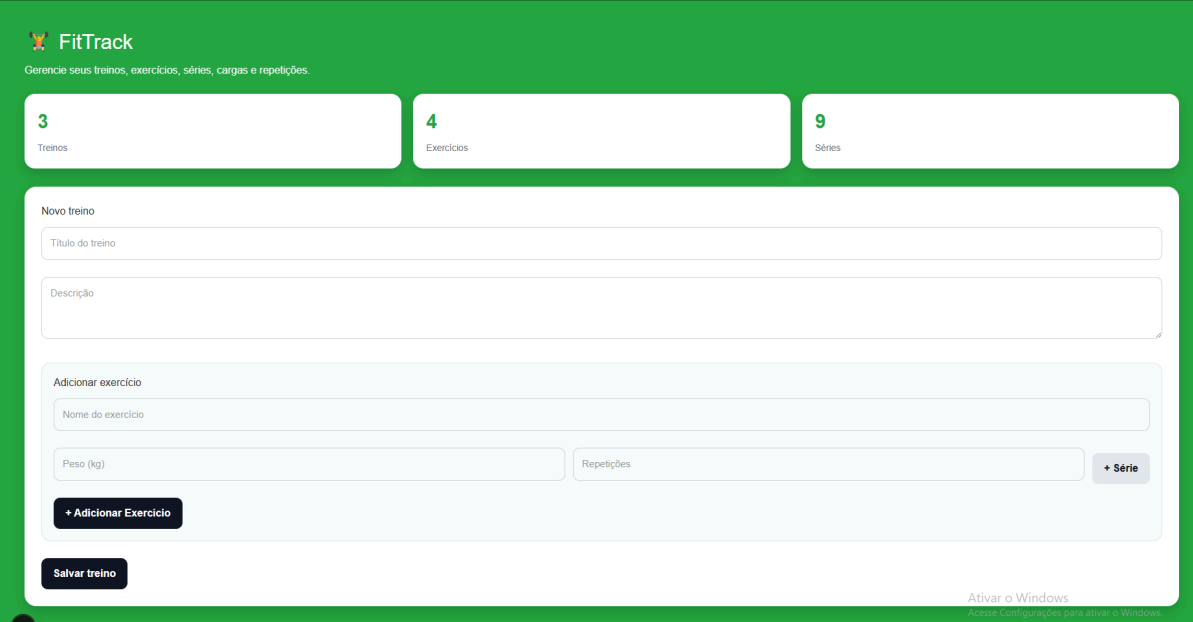
O Dashboard do **FitTrack** atua como a central de inteligência da aplicação para o usuário autenticado. Ele consolida visualmente os dados brutos armazenados no banco de dados, transformando registros em indicadores claros sobre a consistência e a evolução física do praticante, mitigando o problema da fragmentação de históricos.

A interface faz uso extensivo do estado reativo do React. Assim que a tela é carregada, um gancho de efeito (`useEffect`) dispara uma requisição assíncrona para o endpoint

de sumário da API NestJS. Os dados retornados são distribuídos instantaneamente por cards informativos que exibem:

- O somatório acumulado de treinos concluídos pelo usuário.
- O volume total de exercícios cadastrados em sua grade de rotinas.
- Indicadores dinâmicos de frequência semanal.

O design visual preza pelo contraste e pela legibilidade. Em telas de computadores, esses cards organizam-se em uma grade horizontal que flanqueia o menu lateral de navegação. Já em resoluções menores de celulares, o menu se retrai automaticamente para um formato em estilo hambúrguer (*drawer menu*) e os cards informativos se empilham verticalmente, garantindo que o usuário possa bater o olho nos seus números rapidamente enquanto caminha pela academia.



FitTrack

Gerencie seus treinos, exercícios, séries, cargas e repetições.

3 Treinos

4 Exercícios

9 Séries

Novo treino

Título do treino

Descrição

Adicionar exercício

Nome do exercício

Peso (kg)

Repetições

+ Série

+ Adicionar Exercício

Salvar treino

Ativar o Windows

Acesse Configurações para ativar o Windows.

Figura 5: Painel de Controle (Dashboard) principal em ambiente de desktop.



Figura 6: Painel de Controle adaptado e exibido em dispositivo mobile.

4.2.3 Formulário Dinâmico de Cadastro de Treinos em Lote

A interface de criação de treinos representa a materialização técnica das soluções planejadas para as dores do mercado e os requisitos centrais do sistema (**RF004**). O principal diferencial operativo desta tela é eliminar completamente os fluxos burocráticos e repetitivos comuns em softwares comerciais, concentrando a criação de um bloco completo de dados em um único formulário unificado.

Para atingir esse comportamento flexível, a tela faz uso avançado da manipulação de estados aninhados através de arrays dinâmicos no React. A interface disponibiliza botões para adicionar e remover exercícios reativamente na estrutura da página, bem como botões específicos para criar dinamicamente novas séries de repetições e cargas para cada exercício adicionado.

Dessa forma, o usuário consegue construir um esqueleto estruturado e complexo de treino (Treino -> Exercícios -> Séries) sem recarregar a página e sem realizar nenhuma requisição intermediária ao servidor. Ao clicar no botão de submissão, a aplicação consolida toda a árvore de dados populada na memória do cliente e envia um único payload JSON unificado para o backend NestJS através do Axios, que processa a persistência em lote de forma atômica.

Meus treinos

Treino de Peito
Foco em força

03/07/2026

🔥 Supino reto

Série	Peso	Reps
1	40kg	12
2	45kg	10
3	50kg	8

🔥 Crucifixo

Série	Peso	Reps
1	12kg	15
2	14kg	12

Editar Excluir

Figura 7: Formulário dinâmico em lote em ambiente desktop para criação de rotinas complexas.

Novo treino

Título do treino

Descrição

Adicionar exercício

Nome do exercício

Peso (kg)

Repetições

+ Série

+ Adicionar Exercício

Salvar treino

Meus treinos

Figura 8: Formulário de cadastro dinâmico em lote renderizado em visualização mobile.

5. REQUISITOS COMPLEMENTARES E DESIGN TÉCNICO

Este capítulo aborda as decisões de engenharia de software e os padrões técnicos adotados para garantir que o ecossistema do **FitTrack** opere com máxima eficiência, segurança e aderência às melhores práticas de arquitetura web contemporâneas.

5.1 MECANISMOS DE SEGURANÇA IMPLEMENTADOS

A segurança da informação e a proteção da privacidade dos dados dos usuários foram tratadas como pilares centrais desde o primeiro dia de concepção do sistema, atendendo diretamente aos requisitos de integridade de dados (**RNF003** e **RN002**).

- **Criptografia e Hashing de Senhas:** O sistema impede estritamente o armazenamento de credenciais em texto claro na camada de infraestrutura. Ao criar uma conta ou alterar uma senha, o fluxo de dados intercepta o valor e aplica um algoritmo de dispersão criptográfica irreversível (*hashing*) baseado em **BCrypt**, utilizando um fator de custo adaptativo (*salt*). Mesmo em cenários hipotéticos de vazamento da base de dados, as senhas originais permanecem indecifráveis.
- **Controle de Sessão e Autenticação via JWT (JSON Web Tokens):** Uma vez validadas as credenciais do usuário na rota de login, a API NestJS emite um token assinado digitalmente com uma chave privada forte. Este token contém uma carga útil (*payload*) enxuta, incluindo apenas o identificador único do usuário e o tempo de expiração da sessão.
- **Guarda de Rotas (Guards Globais):** No backend NestJS, a segurança é aplicada em nível de infraestrutura e apresentação por meio de *Guards* (**JwtAuthGuard**). Qualquer requisição direcionada a endpoints protegidos (como criação, leitura ou remoção de treinos) deve obrigatoriamente fornecer o token JWT no cabeçalho HTTP sob o esquema **Authorization: Bearer <TOKEN>**. O NestJS decodifica o token, injeta os dados do usuário autenticado no objeto da requisição e barra de forma automatizada qualquer tentativa de acesso não autorizado, garantindo o isolamento completo de escopo de dados (**RN002**).

[Frontend: Axios + JWT] -> (HTTP Header: Bearer) -> [Backend: NestJS JwtAuthGuard] -> [Use Case]

5.2 ESTRATÉGIAS DE PERFORMANCE ADOTADAS

Para cumprir o requisito de baixa latência (**RNF004**), que estipula o processamento de consultas em tempo inferior a 1 segundo, foram aplicadas otimizações tanto no fluxo de dados do backend quanto na renderização de recursos do cliente:

- **Redução do Tráfego de Rede com DTOs de Saída:** A API NestJS faz uso de serializadores para garantir que apenas as informações estritamente necessárias sejam transmitidas no payload JSON de resposta. Campos irrelevantes para o contexto da

view, ou dados sensíveis como o hash da senha do usuário, são expurgados na camada de apresentação, reduzindo o tamanho em bytes de cada requisição.

- **Consultas Otimizadas no Banco de Dados:** Nas operações de leitura do painel de controle e listagem de fichas, a camada de persistência utiliza carregamento focado parametrizado exclusivamente para os relacionamentos necessários. Isso evita o clássico problema de performance conhecido em engenharia de software como "Consulta \$N+1\$", minimizando o número de viagens de ida e volta ao servidor de banco de dados.

5.3 MODELO DE RENDERIZAÇÃO DA APLICAÇÃO WEB

O **FitTrack** adota o modelo de arquitetura baseada em **SPA**, orquestrado pela combinação do React e do Next.js. Nesse ecossistema, o processo de renderização ocorre predominantemente do lado do cliente .

Sob essa abordagem, quando o navegador faz a requisição inicial para a plataforma, o servidor envia um esqueleto HTML mínimo, acompanhado dos pacotes de JavaScript compactados. A partir desse momento, a interface do usuário é montada e atualizada de forma totalmente dinâmica no navegador. As transições entre páginas ocorrem instantaneamente na memória do cliente, sem a necessidade de recarregar o navegador ou requisitar novos arquivos HTML inteiros ao servidor.

Toda a comunicação com as regras de negócio ocorre de maneira assíncrona em segundo plano, por meio de chamadas AJAX/Fetch executadas pela biblioteca Axios. Esse design técnico maximiza a fluidez da aplicação, fornecendo ao praticante de atividades físicas uma experiência de uso extremamente responsiva e similar à de um aplicativo móvel nativo, permitindo que as interações e atualizações visuais ocorram sem engasgos no ambiente de treino.

6. CONSIDERAÇÕES FINAIS

O desenvolvimento do ecossistema **FitTrack** representou uma oportunidade única para consolidar e expandir de forma prática os conhecimentos técnicos absorvidos ao longo da disciplina de **Frameworks Web**. A proposta de conceber uma plataforma *full-stack* escalável, segura e altamente responsiva nos forçou a encarar problemas reais de engenharia de software, indo muito além do desenvolvimento de interfaces simples ou operações básicas de CRUD.

A adoção do **NestJS** no ecossistema backend provou-se uma decisão técnica extremamente acertada. Através de sua arquitetura opinativa, fomos capazes de blindar o núcleo do negócio utilizando os preceitos de **Clean Architecture** e **DDD**, garantindo que as regras de negócio permanecessem isoladas e imunes a acoplamentos tecnológicos desnecessários. Complementarmente, a validação preemptiva de dados via DTOs e a segurança de rotas centralizada por *Guards* de JWT conferiram ao sistema o nível de maturidade exigido para sistemas web modernos.

Do lado do cliente, a sinergia entre **React** e **Next.js** viabilizou a construção de uma interface rica e altamente adaptável. O maior desafio operativo do frontend — o formulário dinâmico com persistência atômica em lote — foi superado com sucesso por meio de uma manipulação avançada e fluida de estados reativos na memória do navegador. Essa abordagem não apenas resolveu o problema da burocracia em aplicativos tradicionais de musculação, mas também garantiu uma usabilidade extremamente ágil durante os treinos diários.

Em suma, o projeto atendeu de forma integral a todos os Requisitos Funcionais, Não Funcionais e Regras de Negócio propostos inicialmente. O FitTrack consolidou-se como um laboratório prático robusto, validando o uso de frameworks web modernos para a construção de arquiteturas limpas e demonstrando o valor de softwares projetados com foco em performance, organização estrutural e foco na experiência real do usuário.